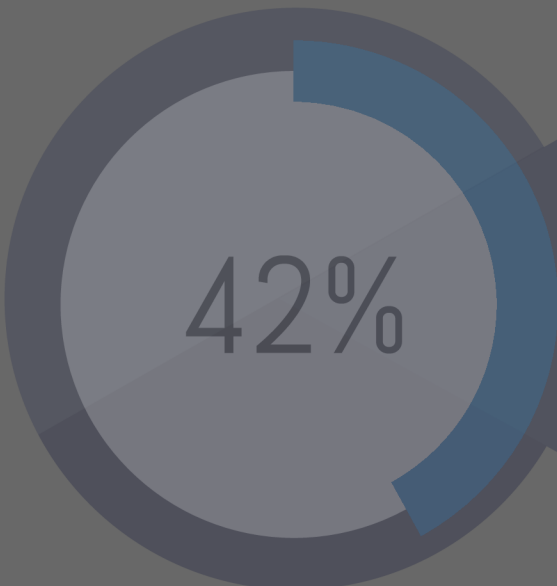


ONLY_Z

Assignment name : only_z
Expected files : only_z.c
Allowed functions: write

Write a program that displays a 'z' character on the standard output.

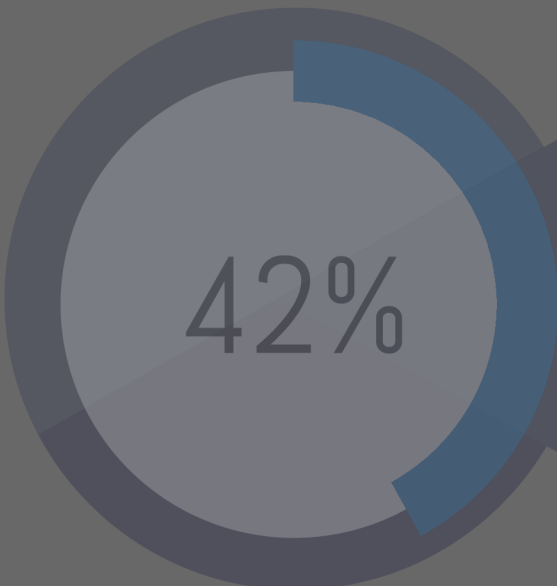


42%

ONLY_A

Assignment name : only_a
Expected files : only_a.c
Allowed functions: write

Write a program that displays a 'a' character on the standard output.



42%

MAFF_REVALPHA

Assignment name : maff_revalpha
Expected files : maff_revalpha.c
Allowed functions: write

Write a program that displays the alphabet in reverse, with even letters in uppercase, and odd letters in lowercase, followed by a newline.

Example:

```
$> ./maff_revalpha | cat -e  
zYxWvUtSrQpOnMlKjIhGfEdCbA$
```

42%

MAFF_ALPHA

Assignment name : maff_alpha
Expected files : maff_alpha.c
Allowed functions: write

Write a program that displays the alphabet, with even letters in uppercase, and odd letters in lowercase, followed by a newline.

Example:

```
$> ./maff_alpha | cat -e  
aBcDeFgHiJkLmNoPqRsTuVwXyZ$
```

42%

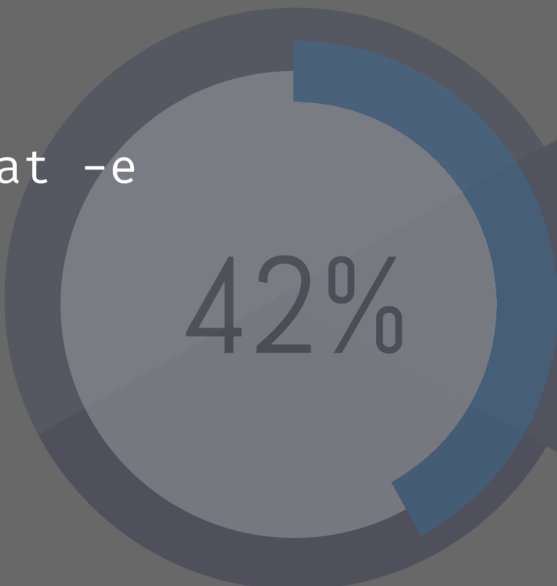
HELLO

Assignment name : hello
Expected files : hello.c
Allowed functions: write

Write a program that displays "Hello World!" followed by a `\n`.

Example:

```
$>./hello
Hello World!
$>./hello | cat -e
Hello World!$
$>
```



42%

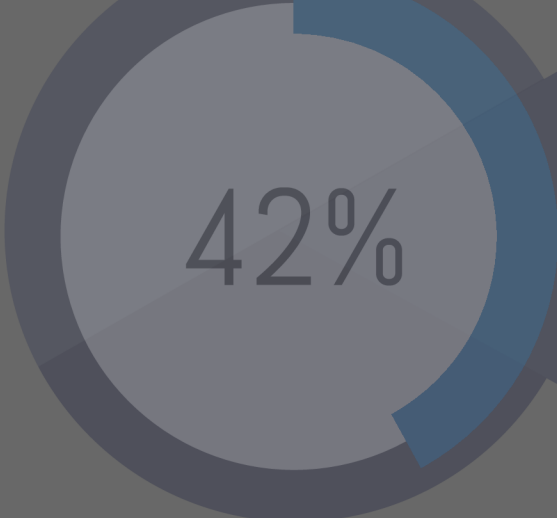
FT_PRINT_NUMBERS

Assignment name : ft_print_numbers
Expected files : ft_print_numbers.c
Allowed functions: write

Write a function that displays all digits in ascending order.

Your function must be declared as follows:

```
void ft_print_numbers(void);
```



42%

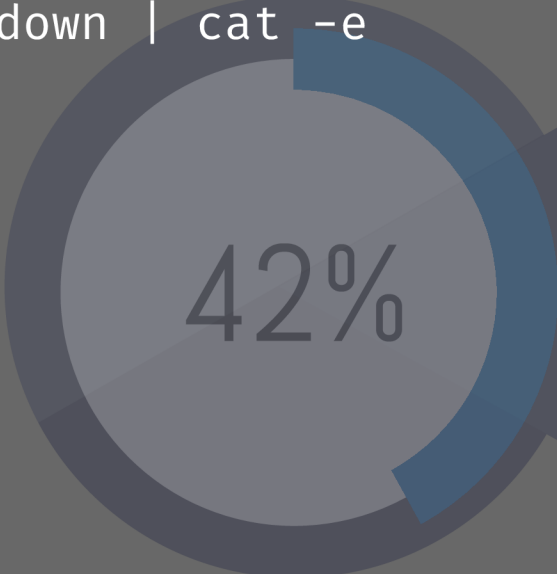
FT_COUNTDOWN

Assignment name : ft_countdown
Expected files : ft_countdown.c
Allowed functions: write

Write a program that displays all digits in descending order, followed by a newline.

Example:

```
$> ./ft_countdown | cat -e  
9876543210$  
$>
```



42%

AFF_Z

Assignment name : aff_z
Expected files : aff_z.c
Allowed functions: write

Write a program that takes a string, and displays the first 'z' character it encounters in it, followed by a newline. If there are no 'z' characters in the string, the program writes 'z' followed by a newline. If the number of parameters is not 1, the program displays 'z' followed by a newline.

Example:

\$> ./aff_z "abc" | cat -e
z\$
\$> ./aff_z "dub0 a POIL" | cat -e
z\$
\$> ./aff_z "zaz sent le poney" | cat -e
z\$
\$> ./aff_z | cat -e
z\$

42%

AFF_FIRST_PARAM

Assignment name : aff_first_param
Expected files : aff_first_param.c
Allowed functions: write

Write a program that takes strings as arguments, and displays its first argument followed by a `\n`.

If the number of arguments is less than 1, the program displays `\n`.

Example:

`$> ./aff_first_param vincent mit "l'ane" dans
un pre et "s'en" vint | cat -e
vincent$
$> ./aff_first_param "j'aime le fromage de
chevre" | cat -e
j'aime le fromage de chevre$
$> ./aff_first_param
$`

AFF_LAST_PARAM

Assignment name : aff_last_param
Expected files : aff_last_param.c
Allowed functions: write

Write a program that takes strings as arguments, and displays its last argument followed by a newline.

If the number of arguments is less than 1, the program displays a newline.

Examples:

```
$> ./aff_last_param "zaz" "mange" "des"  
"chats" | cat -e  
chats$  
$> ./aff_last_param "j'aime le savon" | cat -e  
j'aime le savon$  
$> ./aff_last_param  
$
```

AFF_A

Assignment name : aff_a
Expected files : aff_a.c
Allowed functions: write

Write a program that takes a string, and displays the first 'a' character it encounters in it, followed by a newline. If there are no 'a' characters in the string, the program just writes a newline. If the number of parameters is not 1, the program displays 'a' followed by a newline.

Example:

\$> ./aff_a "abc" | cat -e
a\$
\$> ./aff_a "dub0 a POIL" | cat -e
a\$
\$> ./aff_a "zz sent le poney" | cat -e
\$
\$> ./aff_a | cat -e
a\$

42%